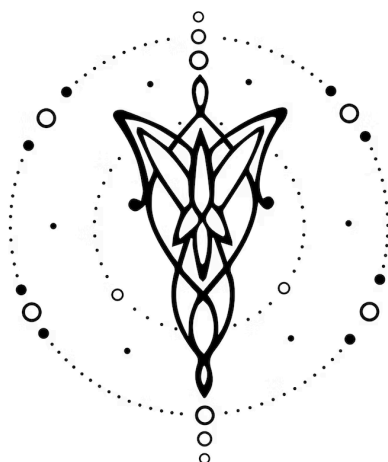




MakeLiteC: Stage III

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

Nombre	Apellido	Legajo	E-mail
Lola	Diaz Varela	62792	ldiazvarela@itba.edu.ar
Federico	Magri	64508	fmagri@itba.edu.ar
Manuel	Cortés Teyssier	64159	mcortesteyssier@itba.edu.ar
Sebastian	Vega Garrot	64111	svegagarrot@itba.edu.ar

FECHA DE ENTREGA: 28 DE NOVIEMBRE 2025

PROFESORES/AYUDANTES:

MARIO AGUSTÍN GOLMAR, ANA MARIA ARIAS ROIG, JOSEFINA GONZALEZ CORNET,

FERNANDO LI, AGUSTÍN TOMÁS ROMERO, LEO WEITZ

ÍNDICE

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1. Dominio.....	3
2.2. Lenguaje.....	3
3. Implementación.....	4
3.1. Frontend.....	4
3.2. Backend.....	4
3.3 Dificultades Encontradas.....	5
4. Futuras Extensiones.....	5
5. Conclusiones.....	6
6. Referencias.....	6
7. Bibliografía.....	6

1. Introducción

El presente informe describe el diseño y desarrollo de MakeLite-C, un lenguaje declarativo de dominio específico (DSL) orientado a la descripción, compilación y ejecución de proyectos escritos en lenguaje C. El objetivo fundamental del proyecto es simplificar la complejidad inherente a la creación de Makefiles tradicionales, los cuales suelen ser propensos a errores de sintaxis (como tabulaciones) y difíciles de mantener en proyectos de escala pequeña y mediana.

MakeLite-C actúa como una capa de abstracción de alto nivel que permite al programador definir la estructura de su proyecto (fuentes, flags de compilación, librerías y fases de construcción) de manera legible y estructurada. El producto final de este proceso no es la compilación directa, sino la generación automática de un Makefile estándar y ejecutable, garantizando que las reglas de dependencia y los comandos del sistema se estructuren correctamente tras pasar por múltiples validaciones semánticas.

2. Modelo Computacional

2.1. Dominio

El dominio del problema se centra en la orquestación del proceso de construcción (build) de software en C. El sistema aborda la necesidad de especificar qué archivos fuente compilar, qué compilador utilizar (ej. gcc, clang), qué flags aplicar, y cómo gestionar librerías y headers.

Un aspecto distintivo del modelo es su enfoque en la seguridad y el control. A diferencia de un script de shell libre, MakeLite-C restringe las acciones permitidas dentro de las fases de construcción a un conjunto seguro de comandos de sistema (*mkdir*, *rm*, *cp*, *mv*). Esto asegura que el proceso de compilación sea predecible y validable antes de generar cualquier salida.

2.2. Lenguaje

El lenguaje MakeLite-C se define por una sintaxis declarativa que promueve la claridad (código "DRY"). Sus características principales incluyen:

- Secciones de Proyecto: Palabras clave reservadas para definir componentes esenciales como **project** (nombre del proyecto), **compiler**, **sources** (archivos .c), **headers** (.h), **libraries** y **output** (nombre del binario).
- Variables e Interpolación: Soporte para la declaración de variables (**var**, **constant**) y su interpolación ($\{\text{VAR}\}$ o use VAR) dentro de listas, facilitando la reutilización de configuraciones como flags o listas de archivos.

- Fases y Comandos: Estructuración del ciclo de vida mediante fases estándar (**pre_build**, **post_build**) y personalizadas (**phase** NAME), donde se ejecutan los comandos permitidos.
- Lógica Condicional: Construcciones de control de flujo (**if fail(...)**, **if success(...)**) que permiten reaccionar al resultado de fases específicas.
- Logging: Configuración nativa para redirigir la salida de la compilación a archivos de registro.

3. Implementación

La arquitectura del compilador se divide estrictamente en un Frontend encargado del análisis y un Backend generador.

3.1. Frontend

El Frontend convierte el código fuente en un Árbol de Sintaxis Abstracta (AST) en C y realiza las primeras validaciones, siendo:

- Análisis Léxico (Flex): Se definen contextos léxicos (INITIAL, COMMANDS, VAR_INTERPOLATION) para manejar la distinción entre la sintaxis del lenguaje y los comandos de sistema embebidos.
- Análisis Sintáctico (Bison): La gramática define las reglas para declaraciones y secciones. Durante el parsing, se construye el AST y se expanden las variables; esto significa que el AST resultante contiene los valores finales interpolados, eliminando las referencias a variables.
- Semántica: Se validan reglas críticas como la obligatoriedad de la sección sources, la declaración previa de variables antes de su uso y la validez de los comandos dentro de las fases.

3.2. Backend

El Backend recibe un AST validado y ejecuta un pipeline de transformación final:

1. Análisis Semántico del Backend: Verifica la coherencia global (existencia de fase única project, validación de referencias en condicionales, configuración de logging).
2. Expansión de Globs: Un módulo dedicado (GlobExpander) detecta patrones como *.c o src/*.c y los resuelve contra el sistema de archivos real, transformando patrones en listas explícitas de archivos en el AST.

3. Generación de Makefile: El MakefileGenerator traduce la estructura del AST a variables de Makefile (CC, CFLAGS, SOURCES) y targets (all, clean, run). Mapea las fases personalizadas y condicionales a objetivos específicos del Makefile, asegurando que la lógica de ejecución (éxito/fallo) se preserve.

3.3 Dificultades Encontradas

El diseño arquitectónico revela desafíos específicos en la gestión de información entre etapas:

- Persistencia de Variables: Debido a que el Frontend resuelve y expande las variables durante el parsing para simplificar el AST, el Backend pierde la noción de la "tabla de símbolos" original. Esto limita la capacidad del Backend para realizar análisis semánticos sobre las declaraciones de variables originales, ya que solo recibe los valores finales (cadenas de texto).
- Validación de Patrones: La expansión de globs (*, ?) requiere interactuar con el sistema de archivos durante la compilación, lo que introduce la posibilidad de errores externos (ej. patrones sin coincidencias) que deben ser manejados como errores semánticos dentro del flujo de compilación.

4. Futuras Extensiones

A continuación se presentan un conjunto de posibles ampliaciones del lenguaje y del compilador que nos gustaría incorporar en futuras versiones de MakeLite-C.

- Comandos del sistema operativo: Actualmente contamos únicamente 4 comandos válidos: mkdir, rm, cp y mv. Sin embargo, sería útil permitir también chmod para modificar permisos de archivos, touch para generar archivos vacíos o actualizar marcas temporales y tar/zip para empaquetar archivos.
- Comandos de texto: En un futuro podría incluirse el comando echo para permitir imprimir mensajes personalizados durante el proceso de compilación, testing, etc.
- Lenguaje: Incorporación de nuevas secciones estándar (test, clean, benchmark) y ampliación del set de comandos permitidos (touch, grep).
- Funciones/Macros: Sería interesante poder definir funciones o macros para tareas repetitivas. Esto permitiría reutilizar las funciones dentro de un mismo archivo o incluso entre distintos proyectos si se habilitara la importación de funciones desde otros archivos MakeLite-C.
- Configuraciones condicionales: Otra extensión importante sería permitir bloques condicionales para adaptar la compilación al sistema operativo, la arquitectura o

incluso parámetros del usuario. Incluso podría extenderse a configuraciones más generales, como un "modo debug":

5. Conclusiones

MakeLite-C logra establecer un Lenguaje de Dominio Específico (DSL) eficaz para la gestión de proyectos en C, abstrayendo la complejidad técnica de los Makefiles detrás de una sintaxis legible y declarativa. La separación de responsabilidades entre el Frontend (validación, expansión de variables y construcción del AST) y el Backend (validación de dominio, resolución de sistema de archivos y generación de código) ha resultado en un sistema robusto y extensible. El resultado final, consideramos que es una herramienta que no solo automatiza la creación de scripts de construcción, sino que introduce capas de seguridad y validación semántica inexistentes en el uso manual de Make.